SIMATS ENGINEERING

CO2_AT3 Error Analysis Task

COURSE NAME: Software Testing **COURSE CODE: CSA3711**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 20%

Sl. No	Questions	Bloom's Level
1	A tester designs test cases for an online shopping system using equivalence partitioning but incorrectly groups input ranges (e.g., valid and invalid inputs mixed in one class). Additionally, boundary values are not considered. Analyze the given test design and identify all errors in equivalence class partitioning and boundary value analysis. Examine the root causes of these mistakes with proper reasoning. Propose corrected test cases with proper classification and boundary coverage. Justify how your corrections improve defect detection effectiveness.	3
2	A QA engineer performs random testing on a banking application and claims high coverage without using any structured techniques. Critical defects in transaction limits and boundary conditions are missed. Evaluate the testing approach and identify errors in strategy and coverage assumptions. Analyze the root causes behind missing critical defects. Suggest appropriate black box testing techniques to correct the approach. Justify how these techniques ensure better test adequacy and reliability.	3
3	A developer performs white box testing but creates an incomplete control flow graph, missing decision nodes and loops, resulting in poor path coverage. Analyze the errors in the control flow graph and path testing approach. Identify the root causes for inadequate test coverage. Propose corrections by constructing a proper control flow representation and identifying all independent paths. Justify how improved path testing ensures better coverage and defect detection.	5

Sl. No	Questions	Bloom's Level
4	A testing team reports 100% statement coverage for a module but fails to detect logical errors in conditional statements. Evaluate the adequacy of the testing performed and identify gaps in coverage criteria. Analyze why statement coverage alone was insufficient. Recommend improved white box testing strategies such as branch or path coverage. Justify how the revised approach enhances overall test effectiveness and error detection.	5
5	A tester uses Equivalence Class Partitioning for an age-validation field (0-100) but defines only two classes: Negative numbers and Positive numbers. Analyze the errors in this partitioning strategy. Identify why this fails to account for valid vs. invalid age limits and the absence of Boundary Value Analysis. Propose a corrected set of partitions and justify how they ensure more robust input validation.	3
6	A developer claims full Condition Coverage for a complex `if(A && B	
7	A QA team applies Decision Table Testing for a discount system but misses several combinations of "Premium Member" and "Coupon Applied," leading to price calculation errors. Analyze the errors in the construction of the decision table. Identify why the current approach is incomplete. Propose a revised, exhaustive decision table and justify its effectiveness in preventing production defects.	3
8	During unit testing of a looping structure, a tester checks only the first and last iterations (n=1 and n=max). Defects occurring at n=0 and n=max+1 are missed. Evaluate the errors in this Loop Testing strategy. Analyze the root causes behind these omissions. Recommend a corrected approach for testing simple and nested loops, and justify how it ensures path integrity.	5
9	An engineer utilizes Error Guessing as the primary testing technique for a safety-critical braking system, ignoring structured black box methods. Identify the errors in relying solely on ad-hoc techniques for high-risk modules. Analyze the impact on test adequacy. Suggest a combination of BVA and State Transition Testing to fix the strategy, justifying the increased safety assurance.	5

Sl. No	Questions	Bloom's Level
10	A tester designs a Control Flow Graph (CFG) for a module but fails to represent the "else" branches of multiple nested if statements, leading to 100% statement coverage but 0% branch coverage on error paths. Analyze the errors in graph construction. Identify the root causes for the missing nodes. Propose a corrected CFG and independent paths to ensure branch adequacy.	5

Code of Conduct:

*I **Mugesh S (192411055)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

ANSWERS

Question 1: Error Analysis in Equivalence Class Partitioning and Boundary Value Analysis

1. Error Identification

Equivalence Class Partitioning (ECP) divides input data into groups that are expected to behave similarly. Boundary Value Analysis (BVA) tests values at the edges of valid and invalid ranges.

The tester makes the following errors:

1. Valid and invalid input ranges are grouped into the same equivalence class.
2. Boundary values are not tested.
3. Representative values are not selected from each valid and invalid class.
4. The test cases do not adequately cover edge conditions, increasing the possibility of undetected defects.

2. Root Cause Analysis

The mistakes occur because the tester has not correctly identified the input requirements and validation limits. Combining valid and invalid inputs in one class assumes that all values behave similarly, even though they should produce different results.

Ignoring boundary values can miss errors involving minimum and maximum limits, incorrect comparison operators, and off-by-one mistakes.

3. Corrected Test Cases

Consider an online shopping system that accepts a product quantity from 1 to 100.

Test Case	Input Quantity	Classification	Expected Result
TC01	0	Invalid — below minimum	Reject quantity
TC02	1	Valid — minimum	Accept quantity
TC03	2	Valid — normal range	Accept quantity
TC04	50	Valid — normal range	Accept quantity
TC05	99	Valid — normal range	Accept quantity
TC06	100	Valid — maximum	Accept quantity
TC07	101	Invalid — above maximum	Reject quantity
TC08	-1	Invalid — negative value	Reject quantity

The equivalence classes are:

- Invalid class 1: Quantity less than 1.
- Valid class: Quantity from 1 to 100.
- Invalid class 2: Quantity greater than 100.

For BVA, the important values are 0, 1, 2, 99, 100, and 101.

4. Justification

Correct ECP reduces the number of test cases while maintaining meaningful input coverage. BVA focuses on values near the limits, where errors frequently occur. Combining both techniques improves defect detection, reduces redundant tests, and increases confidence in the shopping system's input validation.

Question 2: Error Analysis of Random Testing in a Banking Application

1. Error Identification

Random testing selects test inputs without systematically covering important conditions. The QA engineer incorrectly assumes that random inputs guarantee high test coverage.

The major errors are:

1. No systematic selection of test cases.
2. Transaction limits and boundary values are ignored.
3. Critical banking conditions may remain untested.
4. Test coverage is claimed without supporting coverage measurements.
5. There is no clear mapping between requirements, test cases, and expected results.

2. Root Cause Analysis

Random testing may select many ordinary values while rarely selecting critical values such as the minimum transaction amount, maximum permitted amount, or an amount just above the limit.

Banking applications require careful validation because transaction errors can cause financial loss. Without structured techniques, the testing process may overlook invalid inputs, insufficient balances, and transaction-limit violations.

3. Corrected Black-Box Testing Techniques

A. Equivalence Class Partitioning

Divide transaction amounts into valid and invalid classes. For example, if the permitted transaction range is ₹100 to ₹50,000:

- Below ₹100: Invalid.
- ₹100 to ₹50,000: Valid.
- Above ₹50,000: Invalid.

B. Boundary Value Analysis

Test values around both limits.

Test Case	Transaction Amount	Expected Result
TC01	₹99	Reject
TC02	₹100	Accept if balance and other conditions are valid
TC03	₹101	Accept if balance and other conditions are valid
TC04	₹49,999	Accept if balance and other conditions are valid
TC05	₹50,000	Accept if balance and other conditions are valid
TC06	₹50,001	Reject

C. Decision Table Testing

Test combinations of transaction amount, account status, and available balance. A transaction should be processed only when all required conditions are satisfied.

D. State Transition Testing

Test transitions such as initiated, pending, successful, failed, and reversed transactions. Verify that invalid transitions and duplicate processing are prevented.

4. Justification

Structured black-box techniques ensure that valid inputs, invalid inputs, boundary conditions, and combinations of conditions are systematically tested. Random testing may still be used as an additional technique, but it should not replace requirement-based test design and measurable coverage analysis.

Question 3: Error Analysis in Control Flow Graph and Path Testing

1. Error Identification

A Control Flow Graph (CFG) represents the execution flow of a program. Nodes represent statements or basic blocks, and directed edges represent possible control transfers.

The developer makes the following errors:

1. Decision nodes are omitted from the CFG.
2. Loop conditions and loop-back edges are missing.
3. Alternative execution paths are not represented.
4. Independent paths are not identified.
5. Test cases do not exercise all important branches and loop conditions.

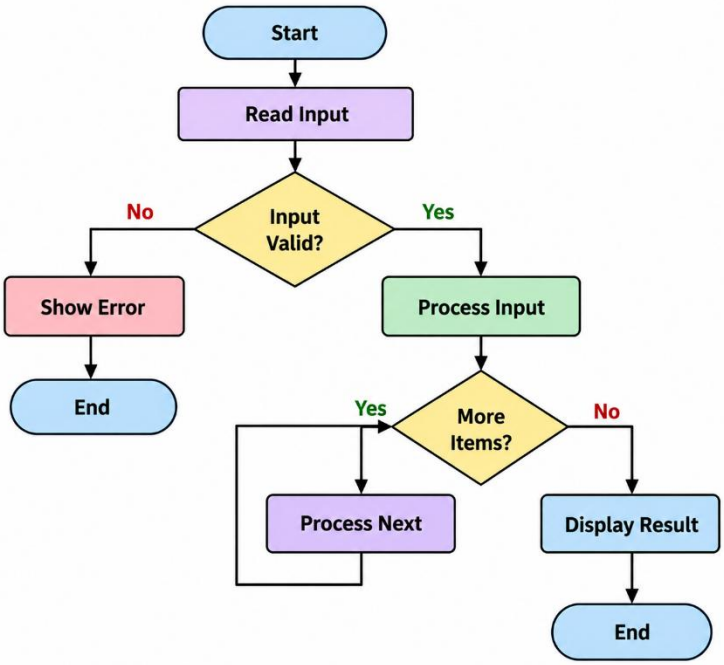
2. Root Cause Analysis

The developer may have constructed the graph without analyzing every conditional statement, loop entry, loop exit, and possible branch. As a result, the graph does not represent the actual program logic.

When loops and decision nodes are omitted, test cases may execute only the main path. Errors in alternative branches, loop termination, and repeated execution can remain undetected.

3. Corrected Control Flow Representation

Consider the following simplified program logic:



The CFG must contain:

- A start node and an end node.
- A node for input validation.
- Both the valid and invalid branches.
- A loop condition and a loop-back edge.
- A loop exit path leading to the final result.

4. Independent Paths

For the simplified program, representative independent paths are:

- Path 1: Start → Read Input → Invalid Input → Show Error → End.
- Path 2: Start → Read Input → Valid Input → No More Items → Display Result → End.
- Path 3: Start → Read Input → Valid Input → More Items → Process Next → Loop Condition → No More Items → Display Result → End.
- Path 4: Start → Read Input → Valid Input → More Items → Process Next → Loop Condition → More Items → Process Next again.

Path 4 illustrates repeated loop execution. Actual independent paths depend on the program's precise CFG and its number of decision nodes.

5. Justification

A complete CFG makes all major control-flow alternatives visible. Independent-path testing ensures that important branches and loop behavior are exercised. It improves defect detection by identifying errors in conditions, loop termination, and execution sequences. For loops, a finite set of tests should cover zero, one, and multiple iterations where applicable.

Question 4: Limitations of 100% Statement Coverage

1. Error Identification

Statement coverage measures whether every executable statement has been executed at least once.

The testing team reports 100% statement coverage but fails to detect logical errors in conditional statements. The main errors are:

1. Statement coverage is treated as proof that the module is fully tested.
2. Both outcomes of conditional statements may not have been exercised.
3. Compound Boolean conditions may not have been tested with different truth values.
4. Important combinations of inputs may be missing.

2. Root Cause Analysis

Statement coverage does not guarantee that every branch of a decision has been executed. A test case may execute a statement inside an if block without testing the alternative else path.

For example:

if age >= 18: print("Eligible")else: print("Not Eligible")

If the only test input is age = 20, the statement print("Eligible") executes, but the else statement does not. Therefore, statement coverage is not 100% in this example because one executable statement remains unexecuted.

However, a more complex program can achieve 100% statement coverage while still failing to test every decision outcome or condition combination.

3. Improved Testing Strategies

A. Branch Coverage

Branch coverage requires each decision outcome to execute at least once.

For the example above:

Test Case	Age	Expected Output
TC01	20	Eligible
TC02	16	Not Eligible

These test cases exercise both the true and false outcomes.

B. Condition Coverage

Condition coverage requires each individual Boolean condition to evaluate to both true and false.

For example:

if age >= 18 and has_id: print("Entry Allowed")else: print("Entry Denied")

Test cases should be designed so that age >= 18 and has_id each evaluate to true and false at least once.

Test Case	Age Condition	Has ID	Expected Output
TC01	True	True	Entry Allowed
TC02	False	True	Entry Denied
TC03	True	False	Entry Denied

C. Path Coverage

Path testing examines different execution paths through the program. Complete path coverage may be impossible for programs with loops or an unbounded number of paths, so practical testing should prioritize feasible and risk-relevant paths.

4. Justification

Statement coverage confirms that statements have executed, but it does not prove that decision logic is correct. Branch coverage detects untested alternatives, condition coverage examines individual Boolean conditions, and path testing examines execution sequences. Combining these techniques provides stronger evidence of software correctness.

Question 5: Error Analysis in Age Validation Using Equivalence Class Partitioning

1. Error Identification

The tester defines only two classes for an age field ranging from 0 to 100:

- Negative numbers.
- Positive numbers.

This classification is incomplete because it does not distinguish valid ages from positive but invalid values.

The errors are:

1. All positive values are treated as one class.
2. Values above 100 are not separated as invalid.
3. The value 0 is not correctly classified if the system accepts ages from 0 to 100.

4. Boundary values are not tested.
5. Invalid data types, such as letters, may be overlooked.

2. Root Cause Analysis

The tester has not used the application's actual validation rule to create equivalence classes. Positive numbers include both valid values, such as 25, and invalid values, such as 150.

The absence of BVA also means that errors at the minimum and maximum permitted ages may remain undetected.

3. Corrected Equivalence Classes

Assume the system accepts integer ages from 0 through 100, inclusive.

Class	Input	Classification	Expected Result
EC1	Less than 0	Invalid	Reject
EC2	0 to 100	Valid	Accept
EC3	Greater than 100	Invalid	Reject
EC4	Non-integer value, such as 25.5	Invalid if only integers are permitted	Reject
EC5	Non-numeric input, such as "ABC"	Invalid	Reject

4. Boundary Value Test Cases

Test Case	Age	Expected Result
TC01	-1	Reject
TC02	0	Accept
TC03	1	Accept
TC04	99	Accept
TC05	100	Accept
TC06	101	Reject
TC07	25.5	Reject if integer ages are required
TC08	ABC	Reject

5. Justification

Correct equivalence classes ensure that valid and invalid inputs are treated separately. Boundary testing checks the values immediately below, at, and above the permitted limits. Together, these methods improve age validation, reduce redundant testing, and help detect incorrect range checks and data-type errors.

Question 6: Error Analysis in Condition Coverage

Note: The uploaded question is incomplete. It ends after the expression if(A && B, so the answer below addresses the condition-coverage issue implied by the available text.

1. Error Identification

Condition coverage requires each individual Boolean condition in a decision to evaluate to both true and false at least once.

A developer incorrectly claims full condition coverage for a complex decision without verifying the outcomes of each individual condition.

For example:

if A and B: print("Accepted")else: print("Rejected")

The decision depends on two conditions, A and B. Testing only the case where both are true does not provide full condition coverage.

2. Root Cause Analysis

The developer may have confused decision coverage with condition coverage. Decision coverage checks whether the overall decision evaluates to true and false. Condition coverage checks whether every individual condition evaluates to true and false.

For compound Boolean expressions, testing only the overall decision result can leave individual conditions insufficiently tested.

3. Corrected Test Cases

For the expression A and B, the following test cases provide condition coverage and exercise both decision outcomes.

Test Case	A	B	Decision Result
TC01	True	True	True
TC02	False	True	False
TC03	True	False	False
TC04	False	False	False

In these test cases, both A and B evaluate to true at least once and false at least once.

4. Improved Testing Strategy

The developer should:

1. Identify every individual Boolean condition.
2. Prepare test cases that make each condition true and false.
3. Verify the overall decision outcome for each test.
4. Use decision coverage in addition to condition coverage.
5. Consider Modified Condition/Decision Coverage (MC/DC) for safety-critical systems where required.

MC/DC requires showing that each condition can independently affect the overall decision outcome while the other conditions are held appropriately fixed.

5. Justification

Correct condition testing exposes errors in Boolean expressions, incorrect logical operators, and missing condition combinations. It provides stronger evidence than testing only the overall true and false outcomes of a decision.

Question 7: Error Analysis in Decision Table Testing for a Discount System

1. Error Identification

Decision Table Testing verifies system behavior for combinations of conditions and their corresponding actions.

The QA team misses combinations of "Premium Member" and "Coupon Applied," resulting in incorrect price calculations.

The major errors are:

1. Some combinations of conditions are missing.
2. The relationship between membership status and coupon usage is not fully analyzed.
3. Expected discount actions are not clearly defined for every combination.
4. Test cases do not cover all relevant rules.
5. The final price is not verified against the expected result for each rule.

2. Root Cause Analysis

The team may have created the decision table without listing all possible combinations of the conditions. When two independent Boolean conditions are considered, there are four possible combinations.

If any combination is omitted, a defect associated with that combination may remain undetected. Incorrect discount calculations can result in overcharging or undercharging customers.

3. Corrected Decision Table

Assume the following business rules:

- Premium members receive a 10% discount.
- Customers who apply a valid coupon receive a 5% discount.
- If both apply, the discounts are combined additively for this example.
- Discounts apply to an eligible product price before taxes.

Rule	Premium Member?	Coupon Applied?	Premium Discount	Coupon Discount	Total Discount
R1	Yes	Yes	10%	5%	15%
R2	Yes	No	10%	0%	10%
R3	No	Yes	0%	5%	5%

R4	No	No	0%	0%	0%

These rules assume that both discounts can be combined. If the actual business requirements prohibit combining discounts, the expected actions must be adjusted accordingly.

4. Test Cases

Assume the eligible product price is ₹1,000 before taxes.

Test Case	Premium Member?	Coupon Applied?	Expected Discount	Expected Price Before Tax
TC01	Yes	Yes	15%	₹850
TC02	Yes	No	10%	₹900
TC03	No	Yes	5%	₹950
TC04	No	No	0%	₹1,000

5. Justification

An exhaustive decision table ensures that every combination of the selected conditions is considered. It improves test completeness, reveals missing business rules, and helps prevent pricing defects. The expected discount and final price should be checked for every rule.

Question 8: Error Analysis in Loop Testing

1. Error Identification

The tester checks only the first and last iterations of a loop, using n = 1 and n = max.

However, defects at n = 0 and n = max + 1 are missed.

The errors are:

1. Zero iterations are not tested.
2. The boundary immediately above the maximum is not tested.
3. Only limited loop behavior is examined.
4. Loop termination conditions may not be fully validated.
5. Nested loops may contain untested interactions between their inner and outer iterations.

2. Root Cause Analysis

The tester has not considered the complete range of loop execution conditions. Bugs may occur when a loop executes zero times, executes once, repeats several times, or exceeds its permitted iteration limit.

Off-by-one errors can cause a loop to execute one extra time or stop one iteration too early. Nested loops require additional attention because defects can occur at the beginning, middle, or end of either loop.

3. Corrected Loop Testing Strategy

A. Simple Loop Testing

For a loop that permits between 1 and max iterations, test the following values where applicable:

Test Case	Number of Iterations	Purpose
TC01	0	Verify behavior when the loop is skipped
TC02	1	Verify the minimum permitted execution
TC03	2	Verify repeated execution
TC04	max − 1	Verify behavior just below the maximum
TC05	max	Verify the maximum permitted execution
TC06	max + 1	Verify that the limit is enforced

If the loop permits zero iterations, the expected result for n = 0 should be normal completion without executing the loop body. If zero is invalid according to the requirements, the system should reject it or handle it as specified.

B. Nested Loop Testing

For nested loops:

1. Test the outer loop at its minimum, typical, and maximum iteration counts.
2. Test the inner loop at its minimum, typical, and maximum iteration counts.
3. Check the inner loop when the outer loop executes zero times, if permitted.
4. Verify the inner loop's initialization and termination for each outer iteration.
5. Test combinations that may cause early termination or boundary errors.

A practical approach is to begin with the innermost loop, test its boundaries, and then gradually extend testing to the outer loops.

4. Justification

Correct loop testing detects off-by-one errors, incorrect termination conditions, unexpected repeated execution, and failures involving zero iterations. Testing simple and nested loops systematically improves control-flow reliability and helps ensure that loops execute the correct number of times.

Question 9: Error Analysis in Error Guessing for a Safety-Critical Braking System

1. Error Identification

Error Guessing uses experience and intuition to predict where defects may occur. However, the engineer relies on it as the primary testing technique for a safety-critical braking system and ignores structured black-box testing.

The errors include:

1. Testing depends excessively on individual experience.

2. Important braking conditions may not be systematically covered.
3. Boundary conditions may be overlooked.
4. Different braking states and transitions may remain untested.
5. There is insufficient evidence that the test cases cover safety-critical requirements.

2. Root Cause Analysis

Error Guessing can identify likely defects, but it cannot guarantee systematic coverage. A braking system must handle different vehicle speeds, sensor readings, braking commands, and system states.

If structured testing is ignored, defects such as incorrect threshold handling, delayed braking, or invalid state transitions may remain undetected. In a safety-critical system, these defects can have serious consequences.

3. Corrected Testing Strategy

A. Boundary Value Analysis

Suppose a braking controller accepts a speed input from 0 to 200 km/h for a particular test interface. This range is an illustrative assumption, not a specification for an actual braking system.

Test Case	Speed Input	Purpose
TC01	-1 km/h	Check invalid negative input handling
TC02	0 km/h	Test minimum boundary
TC03	1 km/h	Test just above minimum
TC04	199 km/h	Test just below maximum
TC05	200 km/h	Test maximum boundary
TC06	201 km/h	Check above-maximum input handling

The expected system response must follow the actual braking-system requirements. Invalid input handling should not disable essential safety functions.

B. State Transition Testing

Consider these illustrative states:

- Idle: No braking command is active.
- Monitoring: Sensors are being evaluated.
- Braking: A braking command has been activated.
- Fault: A fault has been detected.
- Recovery: The system is following an approved recovery procedure.

Representative transitions to test include:

Test Case	Initial State	Event or Condition	Expected State
TC01	Idle	Monitoring begins	Monitoring
TC02	Monitoring	Braking condition detected	Braking
TC03	Monitoring	Sensor fault detected	Fault
TC04	Braking	Braking condition remains active	Braking

TC05	Fault	Approved recovery condition met	Recovery

The permitted transitions and exact expected actions must be defined by the system requirements.

C. Error Guessing as a Supplementary Technique

After structured test cases are prepared, experienced testers can add scenarios involving sensor failures, unexpected input sequences, delayed signals, and repeated braking commands.

4. Justification

Boundary Value Analysis checks behavior near important input limits, while State Transition Testing verifies system behavior across operating states. Combining these structured methods with Error Guessing improves test adequacy and supports stronger safety assurance. Safety-critical systems also require appropriate hazard analysis, traceability, and verification against applicable standards.

Question 10: Error Analysis in Control Flow Graphs with Nested If Statements

1. Error Identification

The tester creates a Control Flow Graph but fails to represent the else branches of multiple nested if statements. This results in incomplete control-flow representation and inadequate branch coverage.

The errors are:

1. Alternative branches are missing from the CFG.
2. Error-handling paths are not represented.
3. Independent paths are not correctly identified.
4. Test cases focus mainly on the successful execution path.
5. Statement coverage is incorrectly treated as sufficient evidence of branch adequacy.

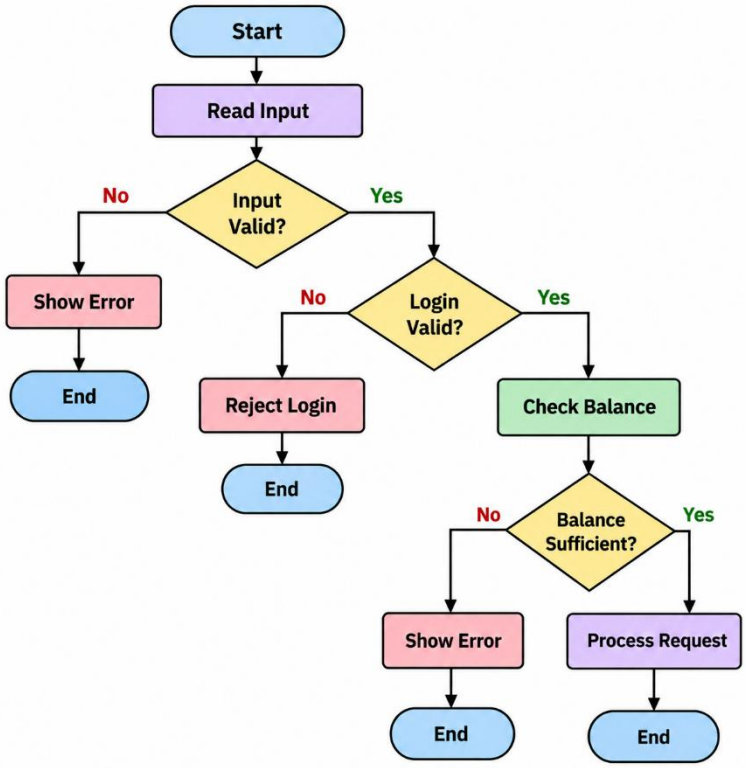
2. Root Cause Analysis

The tester may have considered only the statements executed when conditions are true. However, every decision must represent its possible outcomes, including false outcomes and nested alternatives.

When else branches are omitted, the CFG does not accurately describe the program. As a result, tests may fail to exercise invalid inputs, error-handling statements, or alternative execution paths.

3. Corrected Control Flow Graph

Consider the following simplified nested decision logic:



The corrected CFG must include:

- A decision node for input validation.
- Both true and false branches of input validation.
- A decision node for login validation.
- Both valid-login and invalid-login branches.
- A decision node for balance checking.
- Both sufficient-balance and insufficient-balance branches.
- All error-handling and termination paths.

4. Independent Paths

The following independent paths cover the main alternatives in the simplified example:

- Path 1: Start → Read Input → Invalid Input → Show Error → End.
- Path 2: Start → Read Input → Valid Input → Invalid Login → Reject Login → End.
- Path 3: Start → Read Input → Valid Input → Valid Login → Check Balance → Insufficient Balance → Show Error → End.
- Path 4: Start → Read Input → Valid Input → Valid Login → Check Balance → Sufficient Balance → Process Request → End.

Each path exercises a different combination of decision outcomes.

5. Corrected Test Cases

Test Case	Input Valid?	Login Valid?	Balance Sufficient?	Expected Result
TC01	No	Not evaluated	Not evaluated	Show input error
TC02	Yes	No	Not evaluated	Reject login
TC03	Yes	Yes	No	Show insufficient-balance error
TC04	Yes	Yes	Yes	Process request

The conditions marked “Not evaluated” are not reached because an earlier decision prevents execution from continuing.

6. Justification

A complete CFG represents both the true and false outcomes of each decision. Independent-path testing ensures that important successful and error-handling routes are exercised. This improves branch coverage, helps reveal logical errors, and prevents defects from remaining hidden in nested conditional structures.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand